

UNIVERSITY OF GHANA

Department of Computer Science

Systems Administration (Level 300)

Assignment — Junior Linux System Administrator

Student: Acquah Vincent

Student ID: 22027315

Task 1 — Server Log Investigation

The sample log provided in the assignment was saved locally as `server.log` and used for the tasks below.

```
Last login: Thu Jul 16 21:58:25 on console
ismak@Sneaks-MacBook-Pro ~ % bash

The default interactive shell is now zsh.
To update your account to use zsh, please run 'chsh -s /bin/zsh'.
For more details, please visit https://support.apple.com/kb/HT20858.
ismak@Sneaks-MacBook-Pro ~ % nano server.log
ismak@Sneaks-MacBook-Pro ~ % cat server.log
2026-07-01 08:00:11 INFO User login successful user=paul ip=192.168.1.10
2026-07-01 08:02:44 INFO User login successful user=mary ip=192.168.1.15
2026-07-01 08:04:22 WARNING High CPU usage detected ip=192.168.1.20
2026-07-01 08:06:31 ERROR Database connection failed ip=192.168.1.50
2026-07-01 08:08:12 INFO Backup started ip=192.168.1.30
2026-07-01 08:09:15 ERROR Network timeout ip=192.168.1.50
2026-07-01 08:10:44 INFO Backup completed ip=192.168.1.30
2026-07-01 08:12:19 WARNING Disk usage exceeded 85% ip=192.168.1.22
2026-07-01 08:15:01 INFO User logout user=paul ip=192.168.1.10
2026-07-01 08:18:33 ERROR Authentication failure user=john ip=192.168.1.45
2026-07-01 08:20:11 INFO User login successful user=john ip=192.168.1.45
2026-07-01 08:22:08 INFO File uploaded user=mary ip=192.168.1.15
2026-07-01 08:24:55 WARNING Memory usage above threshold ip=192.168.1.20
2026-07-01 08:26:42 ERROR Disk write failure ip=192.168.1.60
2026-07-01 08:28:15 INFO Service restarted ip=192.168.1.40
2026-07-01 08:30:44 INFO User login successful user=alice ip=192.168.1.18
2026-07-01 08:33:29 ERROR SSL certificate expired ip=192.168.1.18
2026-07-01 08:35:00 INFO Configuration updated ip=192.168.1.25
2026-07-01 08:37:12 WARNING Low disk space ip=192.168.1.22
2026-07-01 08:39:48 INFO Scheduled task completed ip=192.168.1.25
2026-07-01 08:42:33 ERROR Permission denied ip=192.168.1.61
2026-07-01 08:44:10 INFO User logout user=mary ip=192.168.1.15
2026-07-01 08:46:19 WARNING Unusual login detected ip=192.168.1.90
2026-07-01 08:48:55 INFO Backup started ip=192.168.1.30
2026-07-01 08:50:13 INFO Backup completed ip=192.168.1.30
2026-07-01 08:52:41 ERROR File system corruption detected ip=192.168.1.70
2026-07-01 08:54:05 INFO Recovery process initiated ip=192.168.1.70
2026-07-01 08:56:11 WARNING Network latency high ip=192.168.1.50
2026-07-01 08:58:44 INFO User login successful user=daniel ip=192.168.1.17
2026-07-01 09:00:19 ERROR Service unavailable ip=192.168.1.80
2026-07-01 09:02:52 INFO Scheduled maintenance completed ip=192.168.1.25
2026-07-01 09:05:10 INFO User logout user=john ip=192.168.1.45
2026-07-01 09:08:27 WARNING CPU temperature high ip=192.168.1.20
2026-07-01 09:10:46 INFO User login successful user=emma ip=192.168.1.19
2026-07-01 09:12:14 ERROR Access denied ip=192.168.1.61
2026-07-01 09:15:00 INFO System scan completed ip=192.168.1.40
2026-07-01 09:18:18 WARNING Failed login attempts exceeded ip=192.168.1.90
2026-07-01 09:20:35 INFO User logout user=alice ip=192.168.1.18
2026-07-01 09:22:41 ERROR Kernel panic detected ip=192.168.1.99
2026-07-01 09:25:00 INFO System reboot initiated ip=192.168.1.99
ismak@Sneaks-MacBook-Pro ~ %
```

server.log created and verified with `cat server.log`

1.1 — Display every line containing the word ERROR

```
grep "ERROR" server.log
```

```
bash-3.2$ grep "ERROR" server.log
2026-07-01 08:06:31 ERROR Database connection failed ip=192.168.1.50
2026-07-01 08:18:33 ERROR Authentication failure user=john ip=192.168.1.45
2026-07-01 08:26:42 ERROR Disk write failure ip=192.168.1.60
2026-07-01 08:42:33 ERROR Permission denied ip=192.168.1.61
2026-07-01 08:52:41 ERROR File system corruption detected ip=192.168.1.70
2026-07-01 09:00:19 ERROR Service unavailable ip=192.168.1.80
2026-07-01 09:12:14 ERROR Access denied ip=192.168.1.61
2026-07-01 09:22:41 ERROR Kernel panic detected ip=192.168.1.99
bash-3.2$
```

Output: 8 lines containing exact uppercase ERROR

This is a case-sensitive search, so it only matches the literal string "ERROR" and misses the lowercase/mixed-case variants.

1.2 — Display every error message regardless of case (ERROR, Error, error)

```
grep -i "ERROR" server.log
```

```
[bash-3.2$ grep -i "ERROR" server.log
2026-07-01 08:06:31 ERROR Database connection failed ip=192.168.1.50
2026-07-01 08:09:15 error Network timeout ip=192.168.1.50
2026-07-01 08:18:33 ERROR Authentication failure user=john ip=192.168.1.45
2026-07-01 08:26:42 ERROR Disk write failure ip=192.168.1.60
2026-07-01 08:33:29 Error SSL certificate expired ip=192.168.1.18
2026-07-01 08:42:33 ERROR Permission denied ip=192.168.1.61
2026-07-01 08:52:41 ERROR File system corruption detected ip=192.168.1.70
2026-07-01 09:00:19 ERROR Service unavailable ip=192.168.1.80
2026-07-01 09:12:14 ERROR Access denied ip=192.168.1.61
2026-07-01 09:22:41 ERROR Kernel panic detected ip=192.168.1.99
bash-3.2$
```

Output: 10 lines, including 'error' and 'Error' variants

The `-i` flag makes the search case-insensitive, so it also catches "error Network timeout" and "Error SSL certificate expired".

1.3 — Display every ERROR together with its line number

```
grep -in "ERROR" server.log
```

```
[bash-3.2$ grep -in "ERROR" server.log
4:2026-07-01 08:06:31 ERROR Database connection failed ip=192.168.1.50
6:2026-07-01 08:09:15 error Network timeout ip=192.168.1.50
10:2026-07-01 08:18:33 ERROR Authentication failure user=john ip=192.168.1.45
14:2026-07-01 08:26:42 ERROR Disk write failure ip=192.168.1.60
17:2026-07-01 08:33:29 Error SSL certificate expired ip=192.168.1.18
21:2026-07-01 08:42:33 ERROR Permission denied ip=192.168.1.61
26:2026-07-01 08:52:41 ERROR File system corruption detected ip=192.168.1.70
30:2026-07-01 09:00:19 ERROR Service unavailable ip=192.168.1.80
35:2026-07-01 09:12:14 ERROR Access denied ip=192.168.1.61
39:2026-07-01 09:22:41 ERROR Kernel panic detected ip=192.168.1.99
bash-3.2$
```

Output: same 10 matches, each prefixed with its line number

The `-n` flag adds the matching line number before each result.

1.4 — How many ERROR messages exist in the log file?

```
grep -ic "ERROR" server.log
```

```
[bash-3.2$ grep -ic "ERROR" server.log
10
bash-3.2$
```

Output: 10

The `-c` flag counts matching lines instead of printing them. There are 10 ERROR messages in total when all case variants are included (8 if counting strictly uppercase ERROR only).

1.5 — How many WARNING messages exist?

```
grep -ic "WARNING" server.log
```

```
[bash-3.2$ grep -ic "WARNING" server.log
8
bash-3.2$
```

Output: 8

There are 8 WARNING messages in the log file.

1.6 — Display only the unique IP addresses

```
grep -oE 'ip=[0-9.]+' server.log | cut -d= -f2 | sort -u
```

```
[bash-3.2$ grep -oE 'ip=[0-9.]+' server.log | cut -d= -f2 | sort -u
192.168.1.10
192.168.1.15
192.168.1.17
192.168.1.18
192.168.1.19
192.168.1.20
192.168.1.22
192.168.1.25
192.168.1.30
192.168.1.40
192.168.1.45
192.168.1.50
192.168.1.60
192.168.1.61
192.168.1.70
192.168.1.80
192.168.1.90
192.168.1.99
bash-3.2$
```

Output: 19 unique IP addresses, sorted

`grep -oE` extracts each `ip=x.x.x.x` occurrence, `cut` removes the "ip=" prefix, and `sort -u` sorts the results and removes duplicates.

1.7 — Which IP address appears most frequently?

```
grep -oE 'ip=[0-9.]+' server.log | cut -d= -f2 | sort | uniq -c | sort -nr | head -1
```

```
bash-3.2$ grep -oE 'ip=[0-9.]+' server.log | cut -d= -f2 | sort | uniq -c | sort -nr | head -1
4 192.168.1.30
bash-3.2$
```

Output: 192.168.1.30 appears 4 times

Every IP occurrence is extracted (without deduplication), `sort` groups identical IPs together, `uniq -c` counts each group, `sort -nr` orders by count descending, and `head -1` keeps only the top result. 192.168.1.30 is the most frequent IP, appearing 4 times.

1.8 — Why Linux administrators use log files when troubleshooting

Log files record what a system is doing over time — logins, errors, warnings, and resource usage — without requiring an administrator to watch the server live. When a problem occurs, logs let an administrator go back and see exactly when it started, what triggered it, and which processes or IP addresses were involved. This makes it possible to diagnose intermittent or past issues that can't be reproduced on demand. Logs also help spot patterns, such as repeated failed logins or recurring errors

from the same source, which can indicate a deeper fault or a security threat. In short, logs turn invisible system behavior into evidence.

Task 2 — Server Investigation

Server information was gathered using command-line tools available in the terminal.

Commands used

hostname

```
[bash-3.2$ hostname  
Smeaks-MacBook-Pro.local  
bash-3.2$
```

date

```
[bash-3.2$ date  
Mon Jul 20 23:35:48 GMT 2026  
bash-3.2$
```

uname -r

```
Mon Jul 20 23:35:48 GMT 2026  
[bash-3.2$ uname -r  
25.1.0  
bash-3.2$
```

sw_vers

```
[bash-3.2$ sw_vers  
ProductName:      macOS  
ProductVersion:   26.1  
BuildVersion:     25B78  
bash-3.2$
```

uname -m

```
BuildVersion:     25B78  
[bash-3.2$ uname -m  
arm64  
bash-3.2$
```

uptime

```
[bash-3.2$ uptime  
23:37 up 4 days, 1:40, 2 users, load averages: 2.32 2.86 2.83  
bash-3.2$
```

sysctl hw.memsize

```
smeak
[bash-3.2$ sysctl hw.memsize
hw.memsize: 19327352832
bash-3.2$
```

whoami

```
swapspace:
[bash-3.2$ whoami
smeak
bash-3.2$
```

Summary Table

Item	Result
Hostname	Smeaks-MacBook-Pro.local
Date	Mon Jul 20 23:35:48 GMT 2026
Kernel Version	25.1.0
Operating System	macOS 26.1 (Build 25B78)
Architecture	arm64
Uptime	up 4 days, 1:40, 2 users
Memory	18 GB (19327352832 bytes)
Current User	smeak

Reflection

Before upgrading or maintaining a server, an administrator needs to understand exactly what they're working with. The OS and kernel version determine which updates, drivers, or software packages are compatible. CPU architecture affects which binaries or containers will run correctly. Uptime shows stability history and whether a reboot is overdue. Memory and current load indicate whether the server has enough headroom to handle an upgrade without crashing running services. Knowing who is logged in also prevents disrupting active users. Skipping this step risks compatibility failures, downtime, or data loss so gathering system information turns an upgrade into a controlled, predictable process rather than a gamble.

Task 3 — User and Group Administration

Three users (John, Mary, Paulson) and three groups (HR, Finance, IT) were created and users were assigned to their respective departments.

Create the users

```
useradd -m john useradd -m mary useradd -m paulson
```

```
root@5683f00ca4cd:/# useradd -m john
useradd -m mary
useradd -m paulson
root@5683f00ca4cd:/# █
```

Users created with home directories (-m)

Create the groups

```
groupadd HR groupadd Finance groupadd IT
```

```
root@5683f00ca4cd:/# groupadd HR
groupadd Finance
groupadd IT
root@5683f00ca4cd:/# █
```

Groups created

Verify users and groups exist

```
tail -3 /etc/passwd tail -3 /etc/group
```

```
root@5683f00ca4cd:/# tail -3 /etc/passwd
[tail -3 /etc/group
john:x:1001:1001::/home/john:/bin/sh
mary:x:1002:1002::/home/mary:/bin/sh
paulson:x:1003:1003::/home/paulson:/bin/sh
HR:x:1004:
Finance:x:1005:
IT:x:1006:
root@5683f00ca4cd:/# █
```

All three users and three groups confirmed in the system files

Assign each user to the correct group

```
usermod -aG IT john usermod -aG Finance mary usermod -aG HR paulson
```



```
IT:x:1006:
root@5683f00ca4cd:/# usermod -aG IT john
usermod -aG Finance mary
[usermod -aG HR paulson
root@5683f00ca4cd:/# ,
```

Users appended to their department group (-aG preserves existing group memberships)

Verify group membership

```
groups john groups mary groups paulson
```

```
root@5683f00ca4cd:/# groups john
groups mary
groups paulson
john : john IT
mary : mary Finance
paulson : paulson HR
root@5683f00ca4cd:/#
```

John → IT, Mary → Finance, Paulson → HR — confirmed

Reflection

A user represents a single individual account on the system, with its own login credentials, home directory, and unique identity (UID). A group is a collection of users who share a common set of permissions — it doesn't log in itself, but any user added to it inherits whatever access the group has been granted.

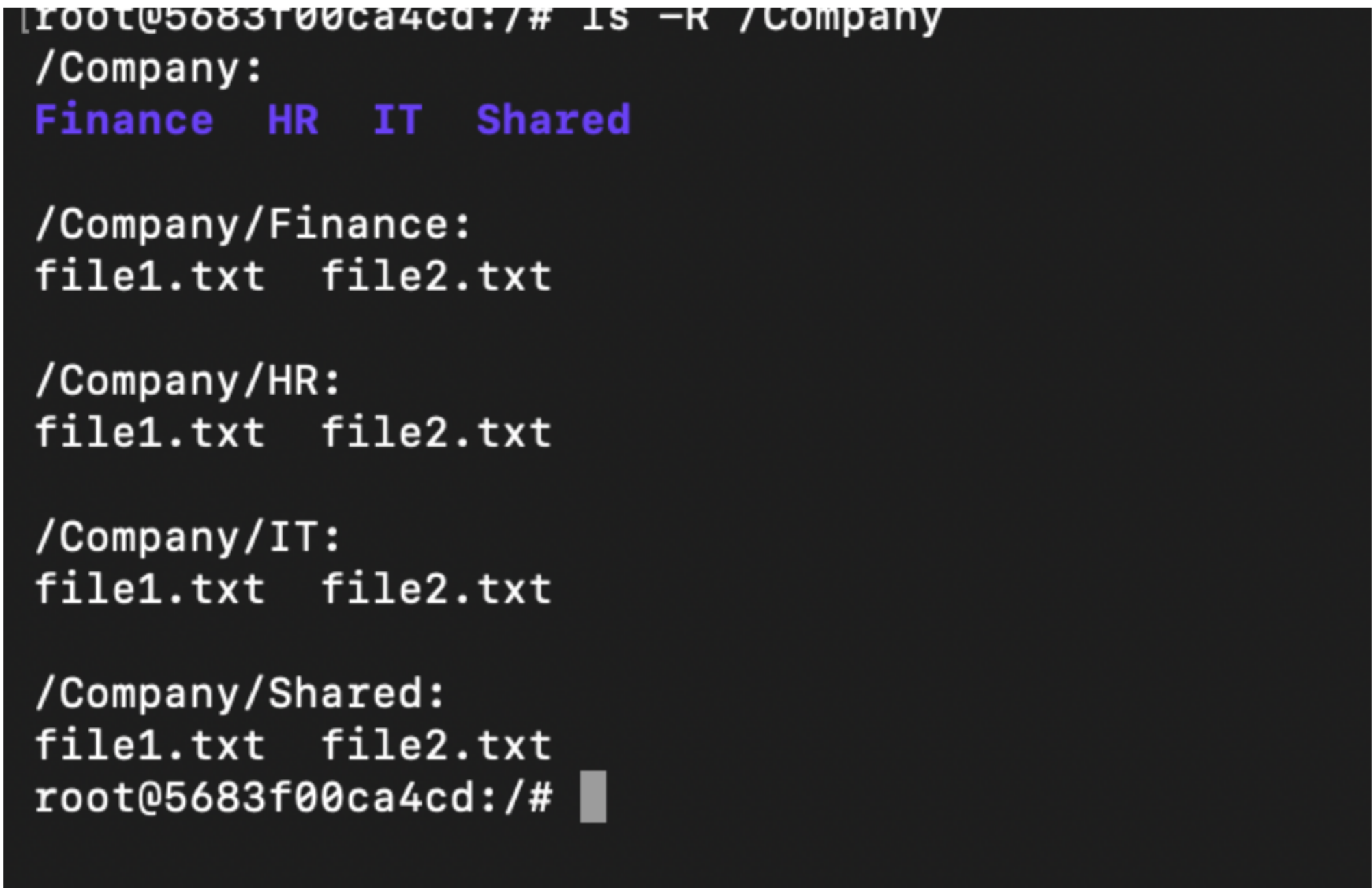
Linux administrators create groups instead of assigning permissions individually because it scales far better. In a company with dozens or hundreds of employees, granting file access one person at a time is slow and error-prone — a single missed user means broken access, and a departing employee means manually revoking permissions everywhere. With groups, the administrator sets permissions once on a shared resource (like the Finance folder) and controls access simply by adding or removing users from the group. It also makes auditing easier: checking who can access Finance files is a matter of listing group members, not searching through every file's individual permission.

Task 4 — Linux File Security

A directory structure was created for Company departments, each containing sample files, with permissions configured according to department access requirements.

Create the directory structure and files

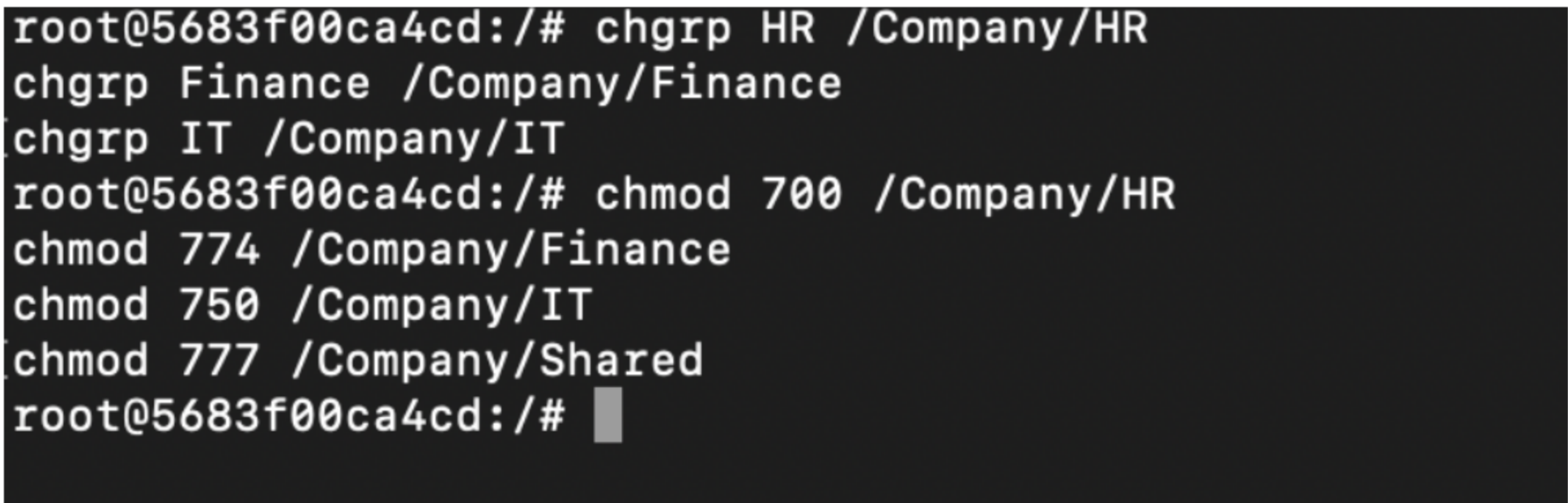
```
mkdir -p /Company/HR /Company/Finance /Company/IT /Company/Shared touch
/Company/HR/file1.txt /Company/HR/file2.txt touch /Company/Finance/file1.txt
/Company/Finance/file2.txt touch /Company/IT/file1.txt /Company/IT/file2.txt
touch /Company/Shared/file1.txt /Company/Shared/file2.txt ls -R /Company
```



Directory structure with two files in each department folder

Set group ownership and permissions

```
chgrp HR /Company/HR chgrp Finance /Company/Finance chgrp IT /Company/IT
chmod 700 /Company/HR chmod 774 /Company/Finance chmod 750 /Company/IT chmod
777 /Company/Shared
```



Group ownership and permission bits applied

Required permissions

Folder	Owner	Group	Others	Octal
--------	-------	-------	--------	-------

HR	Read, write, execute	No access	No access	700
Finance	Full access	Full access	Read only	774
IT	Full access	Read and execute	No access	750
Shared	Read, write, execute	Read, write, execute	Read, write, execute	777

Verify permissions

```
ls -ld /Company/HR /Company/Finance /Company/IT /Company/Shared
```

```
root@5683f00ca4cd:/# ls -ld /Company/HR /Company/Finance /Company/IT /Company/Shared
drwxrwxr-- 2 root Finance 4096 Jul 21 00:49 /Company/Finance
drwx----- 2 root HR      4096 Jul 21 00:49 /Company/HR
drwxr-x--- 2 root IT      4096 Jul 21 00:49 /Company/IT
drwxrwxrwx 2 root root    4096 Jul 21 00:49 /Company/Shared
root@5683f00ca4cd:/#
```

Final permissions confirmed: matches the required table exactly

Reflection

For HR, I set permissions to 700 (owner full, group and others none) because HR files typically contain sensitive personal data — salaries, disciplinary records, personal details — that only HR staff should ever touch. Restricting group and others entirely prevents accidental exposure or tampering by unrelated staff.

For Finance, I used 774 (owner and group full access, others read-only). Finance data needs to be edited by multiple team members collaboratively, so the group needs write access, not just the file owner. Read-only for others allows management or auditors outside Finance to view records without being able to alter them, preserving data integrity.

For IT, I chose 750 (owner full, group read and execute, others none). IT staff often need to run scripts or access configuration files as a team, so execute permission for the group matters, but write access was withheld from the group to prevent accidental changes to shared configurations by any single team member — changes should go through the owner.

For Shared, I used 777 so every department can freely read, write, and access common resources meant for collaboration. This solves the problem of cross-department files being unnecessarily restricted, while the more sensitive department folders stay protected from each other.